

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

AI For Math Seminar, ZJU

Lec 03: Lean AI — instance

Mulikas

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8
- 9 Prop-valued Typeclass

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Mathlib AI

- **instance** Lean

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Mathlib AI

- **instance** Lean
- **Mathlib**

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Mathlib AI

- **instance** Lean
- **Mathlib**
- **AI** LLM Lean

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— instance Lean

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— instance Lean

```
G : Type*    [Group G]    mul_assoc
              Lean
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— instance Lean

```
G : Type*    [Group G]    mul_assoc
              Lean
```



AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— instance Lean

```
G : Type*    [Group G]    mul_assoc
              Lean
```



• [...] Lean instance

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— instance Lean

```
G : Type*    [Group G]    mul_assoc
              Lean
```

-
- [...] Lean instance
- typeclass

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— instance Lean

```
G : Type*    [Group G]    mul_assoc
              Lean
```

-
- [...] Lean instance
- typeclass

```
variable (G : Type*) [Group G]

#synth Group G

#check mul_assoc

#check inv_mul_cancel
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

+ — instance Lean

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

+ — instance Lean

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

+ — instance Lean

- `mul, one, inv`

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

+ — instance Lean

- mul, one, inv
- add, zero, neg

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

+ — instance Lean

- mul, one, inv
- add, zero, neg
- Hom, id, comp


```

Prop-valued Typeclass
  Instance Search
    AI Lean
      AI
        Lean
          Finset
            SimpleGraph

```

+ — instance Lean

- mul, one, inv
- add, zero, neg
- Hom, id, comp

goal

```

Prop-valued Typeclass
  Instance Search
    AI Lean
      AI
        Lean
          Finset
            SimpleGraph

```

+ — instance Lean

- 

goal

```

Prop-valued Typeclass
  Instance Search
    AI Lean
      AI
        Lean
          Finset
            SimpleGraph

```

- mul, one, inv
- add, zero, neg
- Hom, id, comp

●

goal

```
Prop-valued Typeclass
  Instance Search
    AI Lean
      AI
        Lean
          Finset
            SimpleGraph
```

- mul, one, inv
- add, zero, neg
- Hom, id, comp

-
-
-

goal

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

+ — instance Lean

- mul, one, inv
- add, zero, neg
- Hom, id, comp

-
-
-

goal

```
#print      := by
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8
- 9 Prop-valued Typeclass

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

AddGroup / Group — AddGroup Group

AddGroup A	0, +, -a	zero, add, neg
Group G	1, *, g^{-1}	one, mul, inv

```
#check AddGroup
#check Group
#check neg_add_cancel
#check inv_mul_cancel
```

```
example (A : Type*) [AddGroup A] (a : A) :
  -a + a = 0 := neg_add_cancel a
```

```
example (G : Type*) [Group G] (g : G) :
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— AddGroup

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— AddGroup

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— AddGroup

- [Group G]

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— AddGroup

- [Group G]
- $a * b$

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— AddGroup

- [Group G]
- $a * b$
- inv_mul_cancel

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— AddGroup

- [Group G]
- $a * b$
- inv_mul_cancel
-

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— AddGroup

- [Group G]
- $a * b$
- inv_mul_cancel
-

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— AddGroup

- [Group G]

- $a * b$

- inv_mul_cancel

-

- [AddCommGroup V]

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— AddGroup

- [Group G]

- $a * b$

- inv_mul_cancel

-

- [AddCommGroup V]

- $u + v$

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— AddGroup

- [Group G]
- $a * b$
- inv_mul_cancel
-
- [AddCommGroup V]
- $u + v$
- neg_add_cancel

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— AddGroup

- [Group G]
- $a * b$
- inv_mul_cancel
-
- [AddCommGroup V]
- $u + v$
- neg_add_cancel
- [Module K V]

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— AddGroup

- [Group G]
- $a * b$
- inv_mul_cancel
-
- [AddCommGroup V]
- $u + v$
- neg_add_cancel
- [Module K V]

AddGroup

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Monoid — Group

mul	$G \rightarrow G \rightarrow G$	
one		
inv	$G \rightarrow G$	
mul_assoc	cases /	/ group
one_mul, mul_one	rfl	
inv_mul_cancel		

• mul, one, inv —

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Monoid — Group

mul	$G \rightarrow G \rightarrow G$	
one		
inv	$G \rightarrow G$	
mul_assoc	cases /	/ group
one_mul, mul_one	rfl	
inv_mul_cancel		

- mul, one, inv —
- mul_assoc, one_mul, mul_one, inv_mul_cancel —

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Monoid — Group

mul	$G \rightarrow G \rightarrow G$	
one		
inv	$G \rightarrow G$	
mul_assoc	cases /	/ group
one_mul, mul_one	rfl	
inv_mul_cancel		

- mul, one, inv —
- mul_assoc, one_mul, mul_one, inv_mul_cancel —
- Lean instance

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Sign —

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Sign —

```
inductive Sign where
  | pos
  | neg
  deriving DecidableEq, Repr
```

```
open Sign
```

```
instance : Group Sign where
  mul a b :=
    match a, b with
    | pos, x => x
    | neg, pos => neg
    | neg, neg => pos
```


AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Sign —

```
inductive Sign where
  | pos
  | neg
  deriving DecidableEq, Repr
```

```
open Sign
```

```
instance : Group Sign where
  mul a b :=
    match a, b with
    | pos, x => x
    | neg, pos => neg
    | neg, neg => pos
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Sign —

• pos

```
inductive Sign where
  | pos
  | neg
  deriving DecidableEq, Repr
```

```
open Sign
```

```
instance : Group Sign where
  mul a b :=
    match a, b with
    | pos, x => x
    | neg, pos => neg
    | neg, neg => pos
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Sign —

```
inductive Sign where
  | pos
  | neg
  deriving DecidableEq, Repr
```

```
open Sign
```

```
instance : Group Sign where
  mul a b :=
    match a, b with
    | pos, x => x
    | neg, pos => neg
    | neg, neg => pos
```

- pos
- $\text{neg} * \text{neg} = \text{pos}$

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Sign —

```
inductive Sign where
  | pos
  | neg
  deriving DecidableEq, Repr
```

```
open Sign
```

```
instance : Group Sign where
  mul a b :=
    match a, b with
    | pos, x => x
    | neg, pos => neg
    | neg, neg => pos
```

- pos
- $\text{neg} * \text{neg} = \text{pos}$
-

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Sign —

```
inductive Sign where
  | pos
  | neg
  deriving DecidableEq, Repr
```

```
open Sign
```

```
instance : Group Sign where
  mul a b :=
    match a, b with
    | pos, x => x
    | neg, pos => neg
    | neg, neg => pos
```

- pos
- $\text{neg} * \text{neg} = \text{pos}$
-
-

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Sign —

```
inductive Sign where
  | pos
  | neg
  deriving DecidableEq, Repr
```

```
open Sign
```

```
instance : Group Sign where
  mul a b :=
    match a, b with
    | pos, x => x
    | neg, pos => neg
    | neg, neg => pos
```

- pos
- $\text{neg} * \text{neg} = \text{pos}$
-
-

```
pos * neg
neg * neg
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Group

—

cases

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Group — cases

```
mul_assoc := by
  intro a b c
  cases a <;> cases b
    <;> cases c <;> rfl
one_mul := by
  intro a
  cases a <;> rfl
mul_one := by
  intro a
  cases a <;> rfl
inv_mul_cancel := by
  intro a
  cases a <.> rfl
```


AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Group — cases

```
mul_assoc := by
  intro a b c
  cases a <;> cases b
    <;> cases c <;> rfl
one_mul := by
  intro a
  cases a <;> rfl
mul_one := by
  intro a
  cases a <;> rfl
inv_mul_cancel := by
  intro a
  cases a <.> rfl
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Group — cases

```
mul_assoc := by
  intro a b c
  cases a <;> cases b
    <;> cases c <;> rfl
one_mul := by
  intro a
  cases a <;> rfl
mul_one := by
  intro a
  cases a <;> rfl
inv_mul_cancel := by
  intro a
  cases a <.> rfl
```

1 intro

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Group — cases

```
mul_assoc := by
  intro a b c
  cases a <;> cases b
    <;> cases c <;> rfl
one_mul := by
  intro a
  cases a <;> rfl
mul_one := by
  intro a
  cases a <;> rfl
inv_mul_cancel := by
  intro a
  cases a <.> rfl
```

- 1 intro
- 2 cases pos neg

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Group — cases

```
mul_assoc := by
  intro a b c
  cases a <.> cases b
    <.> cases c <.> rfl
one_mul := by
  intro a
  cases a <.> rfl
mul_one := by
  intro a
  cases a <.> rfl
inv_mul_cancel := by
  intro a
  cases a <.> rfl
```

- ① intro
- ② cases pos neg
- ③ rfl

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Group — cases

```
mul_assoc := by
  intro a b c
  cases a <.> cases b
    <.> cases c <.> rfl
one_mul := by
  intro a
  cases a <.> rfl
mul_one := by
  intro a
  cases a <.> rfl
inv_mul_cancel := by
  intro a
  cases a <.> rfl
```

- ① intro
- ② cases pos neg
- ③ rfl
- ④ Group

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Group — cases

```
mul_assoc := by
  intro a b c
  cases a <.> cases b
    <.> cases c <.> rfl
one_mul := by
  intro a
  cases a <.> rfl
mul_one := by
  intro a
  cases a <.> rfl
inv_mul_cancel := by
  intro a
  cases a <.> rfl
```

- ① intro
- ② cases pos neg
- ③ rfl
- ④ Group

+ cases + rfl

=

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8
- 9 Prop-valued Typeclass

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— Subtype

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— Subtype

```
def NonzeroInt :=  
  {n :   // n   0}  
  
example : NonzeroInt :=  
  1, by norm_num  
  
example (x : NonzeroInt) :   :=  
  x.1  
  
example (x : NonzeroInt) :  
  x.1   0 := x.property
```

```
#check Subtype.val
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— Subtype

```
def NonzeroInt :=  
  {n :   // n   0}  
  
example : NonzeroInt :=  
  1, by norm_num  
  
example (x : NonzeroInt) :   :=  
  x.1  
  
example (x : NonzeroInt) :  
  x.1   0 := x.property
```

```
#check Subtype.val
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— Subtype

```
def NonzeroInt :=  
  {n : // n 0}
```

• x.1

```
example : NonzeroInt :=  
  1, by norm_num
```

```
example (x : NonzeroInt) : :=  
  x.1
```

```
example (x : NonzeroInt) :  
  x.1 0 := x.property
```

```
#check Subtype.val
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— Subtype

```
def NonzeroInt :=  
  {n : // n 0}
```

```
example : NonzeroInt :=  
  1, by norm_num
```

```
example (x : NonzeroInt) : :=  
  x.1
```

```
example (x : NonzeroInt) :  
  x.1 0 := x.property
```

```
#check Subtype.val
```

- x.1
- x.2 x.property

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— Subtype

```
def NonzeroInt :=  
  {n : // n 0}
```

```
example : NonzeroInt :=  
  1, by norm_num
```

```
example (x : NonzeroInt) : :=  
  x.1
```

```
example (x : NonzeroInt) :  
  x.1 0 := x.property
```

```
#check Subtype.val
```

- x.1
- x.2 x.property
- value, proof

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— Subtype

```
def NonzeroInt :=  
  {n : // n 0}
```

```
example : NonzeroInt :=  
  1, by norm_num
```

```
example (x : NonzeroInt) : :=  
  x.1
```

```
example (x : NonzeroInt) :  
  x.1 0 := x.property
```

```
#check Subtype.val
```

- x.1
- x.2 x.property
- value, proof
-

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— Subtype

```
def NonzeroInt :=  
  {n : // n 0}
```

```
example : NonzeroInt :=  
  1, by norm_num
```

```
example (x : NonzeroInt) : :=  
  x.1
```

```
example (x : NonzeroInt) :  
  x.1 0 := x.property
```

```
#check Subtype.val
```

- x.1
- x.2 x.property
- value, proof
-

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Group — Subgroup = carrier +

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Group — Subgroup = carrier +

```
def oneSubgroup (G : Type*)  
  [Group G] : Subgroup G  
  where  
    carrier := {g | g = 1}  
    one_mem' := by rfl  
    mul_mem' := by  
      intro a b ha hb  
      rw [ha, hb]; simp  
    inv_mem' := by  
      intro a ha  
      rw [ha]; simp
```

```
example (G : Type*) [Group G]
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Group — Subgroup = carrier +

```
def oneSubgroup (G : Type*)  
  [Group G] : Subgroup G  
  where  
    carrier := {g | g = 1}  
    one_mem' := by rfl  
    mul_mem' := by  
      intro a b ha hb  
      rw [ha, hb]; simp  
    inv_mem' := by  
      intro a ha  
      rw [ha]; simp
```

```
example (G : Type*) [Group G]
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Group — Subgroup = carrier +

```
def oneSubgroup (G : Type*)  
  [Group G] : Subgroup G  
  where  
    carrier := {g | g = 1}  
    one_mem' := by rfl  
    mul_mem' := by  
      intro a b ha hb  
      rw [ha, hb]; simp  
    inv_mem' := by  
      intro a ha  
      rw [ha]; simp
```

① carrier g : G

```
example (G : Type*) [Group G]
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Group — Subgroup = carrier +

```
def oneSubgroup (G : Type*)  
  [Group G] : Subgroup G  
  where  
    carrier := {g | g = 1}  
    one_mem' := by rfl  
    mul_mem' := by  
      intro a b ha hb  
      rw [ha, hb]; simp  
    inv_mem' := by  
      intro a ha  
      rw [ha]; simp
```

- ① carrier g : G
- ② one_mem'

```
example (G : Type*) [Group G]
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Group — Subgroup = carrier +

```
def oneSubgroup (G : Type*)  
  [Group G] : Subgroup G  
  where  
    carrier := {g | g = 1}  
    one_mem' := by rfl  
    mul_mem' := by  
      intro a b ha hb  
      rw [ha, hb]; simp  
    inv_mem' := by  
      intro a ha  
      rw [ha]; simp
```

- ① carrier g : G
- ② one_mem'
- ③ mul_mem'

```
example (G : Type*) [Group G]
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Group — Subgroup = carrier +

```
def oneSubgroup (G : Type*)  
  [Group G] : Subgroup G  
  where  
    carrier := {g | g = 1}  
    one_mem' := by rfl  
    mul_mem' := by  
      intro a b ha hb  
      rw [ha, hb]; simp  
    inv_mem' := by  
      intro a ha  
      rw [ha]; simp
```

- ① carrier g : G
- ② one_mem'
- ③ mul_mem'
- ④ inv_mem'

```
example (G : Type*) [Group G]
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Group — Subgroup = carrier +

```
def oneSubgroup (G : Type*)  
  [Group G] : Subgroup G  
  where  
    carrier := {g | g = 1}  
    one_mem' := by rfl  
    mul_mem' := by  
      intro a b ha hb  
      rw [ha, hb]; simp  
    inv_mem' := by  
      intro a ha  
      rw [ha]; simp
```

- ① carrier g : G
- ② one_mem'
- ③ mul_mem'
- ④ inv_mem'
- ⑤ Mathlib

```
example (G : Type*) [Group G]
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Group — Subgroup = carrier +

```
def oneSubgroup (G : Type*)  
  [Group G] : Subgroup G  
  where  
    carrier := {g | g = 1}  
    one_mem' := by rfl  
    mul_mem' := by  
      intro a b ha hb  
      rw [ha, hb]; simp  
    inv_mem' := by  
      intro a ha  
      rw [ha]; simp
```

- ① carrier g : G
- ② one_mem'
- ③ mul_mem'
- ④ inv_mem'
- ⑤ Mathlib

```
example (G : Type*) [Group G]
```

Subgroup

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing**
- 5 Field
- 6 Module / Submodule
- 7
- 8
- 9 Prop-valued Typeclass

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

PairInt — Ring instance

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

PairInt — Ring instance

```
@[ext]
structure PairInt where
  x : Int
  y : Int

def equivProd : PairInt → Int × Int
  where
    toFun p      := (p.x, p.y)
    invFun q     := q.1, q.2
    left_inv     := by intro p; ext
                  <=> rfl
    right_inv    := by intro q; cases
      q; rfl
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

PairInt — Ring instance

```
@[ext]
structure PairInt where
  x : Int
  y : Int

def equivProd : PairInt → Int × Int
  where
    toFun p      := (p.x, p.y)
    invFun q     := q.1, q.2
    left_inv     := by intro p; ext
                  <=> rfl
    right_inv    := by intro q; cases
      q; rfl
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

PairInt — Ring instance

```
@[ext]
structure PairInt where
  x : Int
  y : Int

def equivProd : PairInt → Int × Int
  where
    toFun p      := (p.x, p.y)
    invFun q     := q.1, q.2
    left_inv     := by intro p; ext
                  <=> rfl
    right_inv    := by intro q; cases
      q; rfl
```

• $\text{Int} \times \text{Int}$

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

PairInt — Ring instance

```
@[ext]
structure PairInt where
  x : Int
  y : Int

def equivProd : PairInt → Int × Int
  where
    toFun p      := (p.x, p.y)
    invFun q     := q.1, q.2
    left_inv     := by intro p; ext
                  <=> rfl
    right_inv    := by intro q; cases
      q; rfl
```

- $\text{Int} \times \text{Int}$
- PairInt

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

PairInt — Ring instance

```
@[ext]
structure PairInt where
  x : Int
  y : Int

def equivProd : PairInt → Int × Int
  where
    toFun p      := (p.x, p.y)
    invFun q     := q.1, q.2
    left_inv     := by intro p; ext
                  <=> rfl
    right_inv    := by intro q; cases
      q; rfl
```

- $\text{Int} \times \text{Int}$
- PairInt
- `Equiv.commRing`

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

PairInt — Ring instance

```
@[ext]
structure PairInt where
  x : Int
  y : Int

def equivProd : PairInt → Int × Int
  where
    toFun p      := (p.x, p.y)
    invFun q     := q.1, q.2
    left_inv     := by intro p; ext
                  <=> rfl
    right_inv    := by intro q; cases
      q; rfl
```

- $\text{Int} \times \text{Int}$
- PairInt
- Equiv.commRing
-

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

#check — Ring

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

#check — Ring

```
#synth CommRing PairInt
#check add_assoc
#check neg_add_cancel
#check left_distrib
#check right_distrib
#check mul_assoc
#check mul_comm
example (p q r : PairInt) :
  p * (q + r) = p * q + p * r :=
  by
    exact mul_add p q r
example (p q : PairInt) :
  (p + q)^2 = p^2 + 2*p*q + q^2
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

#check — Ring

```
#synth CommRing PairInt
#check add_assoc
#check neg_add_cancel
#check left_distrib
#check right_distrib
#check mul_assoc
#check mul_comm
example (p q r : PairInt) :
  p * (q + r) = p * q + p * r :=
  by
    exact mul_add p q r
example (p q : PairInt) :
  (p + q)^2 = p^2 + 2*p*q + q^2
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

#check — Ring

```
#synth CommRing PairInt
#check add_assoc
#check neg_add_cancel
#check left_distrib
#check right_distrib
#check mul_assoc
#check mul_comm
example (p q r : PairInt) :
  p * (q + r) = p * q + p * r :=
  by
    exact mul_add p q r
example (p q : PairInt) :
  (p + q)^2 = p^2 + 2*p*q + q^2
```

1 #synth

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

#check — Ring

```
#synth CommRing PairInt
#check add_assoc
#check neg_add_cancel
#check left_distrib
#check right_distrib
#check mul_assoc
#check mul_comm
example (p q r : PairInt) :
  p * (q + r) = p * q + p * r :=
  by
    exact mul_add p q r
example (p q : PairInt) :
  (p + q)^2 = p^2 + 2*p*q + q^2
```

① #synth

② #check

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

#check — Ring

```
#synth CommRing PairInt
#check add_assoc
#check neg_add_cancel
#check left_distrib
#check right_distrib
#check mul_assoc
#check mul_comm
example (p q r : PairInt) :
  p * (q + r) = p * q + p * r :=
  by
    exact mul_add p q r
example (p q : PairInt) :
  (p + q)^2 = p^2 + 2*p*q + q^2
```

① #synth

② #check

③

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

#check — Ring

```
#synth CommRing PairInt
#check add_assoc
#check neg_add_cancel
#check left_distrib
#check right_distrib
#check mul_assoc
#check mul_comm
example (p q r : PairInt) :
  p * (q + r) = p * q + p * r :=
  by
    exact mul_add p q r
example (p q : PairInt) :
  (p + q)^2 = p^2 + 2*p*q + q^2
```

- 1 #synth
- 2 #check
- 3
- 4 ring

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Ring — Subring Subgroup

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Ring — Subring Subgroup

```
def diagonalSubring : Subring ( ×
  ) where
  carrier := {p | p.1 = p.2}
  zero_mem' := by simp
  one_mem' := by simp
  add_mem' := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
  neg_mem' := by
    intro a ha; simp at ha ; rw
    [ha]
  mul_mem' := by
    intro a b ha hb
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Ring — Subring Subgroup

```
def diagonalSubring : Subring ( ×
  ) where
  carrier := {p | p.1 = p.2}
  zero_mem' := by simp
  one_mem' := by simp
  add_mem' := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
  neg_mem' := by
    intro a ha; simp at ha ; rw
      [ha]
  mul_mem' := by
    intro a b ha hb
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Ring — Subring Subgroup

```
def diagonalSubring : Subring ( ×
  ) where
  carrier := {p | p.1 = p.2}
  zero_mem' := by simp
  one_mem' := by simp
  add_mem' := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
  neg_mem' := by
    intro a ha; simp at ha ; rw
    [ha]
  mul_mem' := by
    intro a b ha hb
```

• $p.1 = p.2$ pair

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Ring — Subring Subgroup

```
def diagonalSubring : Subring ( ×
  ) where
  carrier := {p | p.1 = p.2}
  zero_mem' := by simp
  one_mem' := by simp
  add_mem' := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
  neg_mem' := by
    intro a ha; simp at ha ; rw
    [ha]
  mul_mem' := by
    intro a b ha hb
```

- $p.1 = p.2$ pair
- zero_mem'
- one_mem'

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Ring — Subring Subgroup

```
def diagonalSubring : Subring ( ×
  ) where
  carrier := {p | p.1 = p.2}
  zero_mem' := by simp
  one_mem' := by simp
  add_mem' := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
  neg_mem' := by
    intro a ha; simp at ha ; rw
    [ha]
  mul_mem' := by
    intro a b ha hb
```

- $p.1 = p.2$ pair
- zero_mem'
- one_mem'
- add_mem', neg_mem', mul_mem'

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Ring — Subring Subgroup

```
def diagonalSubring : Subring ( ×
  ) where
  carrier := {p | p.1 = p.2}
  zero_mem' := by simp
  one_mem' := by simp
  add_mem' := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
  neg_mem' := by
    intro a ha; simp at ha ; rw
    [ha]
  mul_mem' := by
    intro a b ha hb
```

- $p.1 = p.2$ pair
- zero_mem'
- one_mem'
- add_mem', neg_mem', mul_mem'
- x.property

```

Prop-valued Typeclass
  Instance Search
    AI Lean
      AI
        Lean
          Finset
            SimpleGraph

```

— Subring Subgroup

- `p.1 = p.2` pair
- `zero_mem'`
`one_mem'`
- `add_mem', neg_mem',`
`mul_mem'`
- `x.property`

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field**
- 6 Module / Submodule
- 7
- 8
- 9 Prop-valued Typeclass

AddGroup / Group
 Subtype
 Ring / CommRing
Field
 Module / Submodule

Prop-valued Typeclass
 Instance Search
 AI Lean
 AI
 Lean
 Finset
 SimpleGraph

— Field = +

	Lean
CommRing K	ring
Nontrivial K	0 1
inv, div	$x / y = x * y^{-1}$
mul_inv_cancel	$x \neq 0$
inv_zero	Lean $0^{-1} = 0$
AI	$x / x = 1$ $x \neq 0$

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

WrappedQ — Rat Field

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

WrappedQ — Rat Field

```
@[ext]
structure WrappedQ where
  val : Rat

def equivRat : Equiv WrappedQ Rat
  where
    toFun x    := x.val
    invFun q   := { val := q }
    left_inv  := by intro x; ext; rfl
    right_inv := by intro q; rfl

instance : Field WrappedQ :=
  equivRat.field
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

WrappedQ — Rat Field

```
@[ext]
structure WrappedQ where
  val : Rat

def equivRat : Equiv WrappedQ Rat
  where
    toFun x    := x.val
    invFun q   := { val := q }
    left_inv  := by intro x; ext; rfl
    right_inv := by intro q; rfl

instance : Field WrappedQ :=
  equivRat.field
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

WrappedQ — Rat Field

```
@[ext]
structure WrappedQ where
  val : Rat

def equivRat : Equiv WrappedQ Rat
  where
    toFun x    := x.val
    invFun q   := { val := q }
    left_inv  := by intro x; ext; rfl
    right_inv := by intro q; rfl

instance : Field WrappedQ :=
  equivRat.field
```

● Rat

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

WrappedQ — Rat Field

```
@[ext]
structure WrappedQ where
  val : Rat

def equivRat : Equiv WrappedQ Rat
  where
    toFun x := x.val
    invFun q := { val := q }
    left_inv := by intro x; ext; rfl
    right_inv := by intro q; rfl

instance : Field WrappedQ :=
  equivRat.field
```

- Rat
-

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

WrappedQ — Rat Field

```
@[ext]
structure WrappedQ where
  val : Rat

def equivRat : Equiv WrappedQ Rat
  where
    toFun x    := x.val
    invFun q   := { val := q }
    left_inv  := by intro x; ext; rfl
    right_inv := by intro q; rfl

instance : Field WrappedQ :=
  equivRat field
```

- Rat
-
- field_simp [hx]

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

WrappedQ — Rat Field

```
@[ext]
structure WrappedQ where
  val : Rat

def equivRat : Equiv WrappedQ Rat
  where
    toFun x := x.val
    invFun q := { val := q }
    left_inv := by intro x; ext; rfl
    right_inv := by intro q; rfl

instance : Field WrappedQ :=
  equivRat field
```

- Rat
-
- field_simp [hx]
- statement

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule**
- 7
- 8
- 9 Prop-valued Typeclass

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Module Lean

Lean

```
[Semiring R] [Field K]
[AddCommGroup V]
r • v [Module R V]
smul_add, add_smul, mul_smul, one_smul
```

```
variable {K : Type*} [Field K]
variable {V : Type*} [AddCommGroup V] [Module K V]
#check smul_add      --  $r \bullet (v + w) = r \bullet v + r \bullet w$ 
#check add_smul      --  $(r + s) \bullet v = r \bullet v + s \bullet v$ 
#check mul_smul      --  $(r * s) \bullet v = r \bullet (s \bullet v)$ 
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Module — Submodule subtype

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Module — Submodule subtype

```
def diagonalSubmodule :  
  Submodule (α × α) where  
  carrier      := {p | p.1 = p.2}  
  zero_mem'    := by simp  
  add_mem'     := by  
    intro a b ha hb  
    simp at ha hb ; rw [ha, hb]  
  smul_mem'    := by  
    intro c p hp  
    change (c * p.1 = c * p.2)  
    rw [hp]  
example (x : diagonalSubmodule) :  
  (x : α × α) 1 = (x : α × α) 2 :=
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Module — Submodule subtype

```
def diagonalSubmodule :  
  Submodule ( × ) where  
  carrier      := {p | p.1 = p.2}  
  zero_mem'    := by simp  
  add_mem'     := by  
    intro a b ha hb  
    simp at ha hb ; rw [ha, hb]  
  smul_mem'    := by  
    intro c p hp  
    change (c * p.1 = c * p.2)  
    rw [hp]  
example (x : diagonalSubmodule) :  
  (x : × ) 1 = (x : × ) 2 :=
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Module — Submodule subtype

```
def diagonalSubmodule :  
  Submodule (x) where  
  carrier := {p | p.1 = p.2}  
  zero_mem' := by simp  
  add_mem' := by  
    intro a b ha hb  
    simp at ha hb ; rw [ha, hb]  
  smul_mem' := by  
    intro c p hp  
    change (c * p.1 = c * p.2)  
    rw [hp]
```

```
example (x : diagonalSubmodule) :  
  (x : x) 1 = (x : x) 2 :=
```

1 carrier

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Module — Submodule subtype

```
def diagonalSubmodule :  
  Submodule ( × ) where  
  carrier      := {p | p.1 = p.2}  
  zero_mem'    := by simp  
  add_mem'     := by  
    intro a b ha hb  
    simp at ha hb ; rw [ha, hb]  
  smul_mem'    := by  
    intro c p hp  
    change (c * p.1 = c * p.2)  
    rw [hp]
```

```
example (x : diagonalSubmodule) :  
  (x : × ) 1 = (x : × ) 2 :=
```

- ① carrier
- ② zero_mem'

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Module — Submodule subtype

```
def diagonalSubmodule :  
  Submodule (α × α) where  
  carrier      := {p | p.1 = p.2}  
  zero_mem'    := by simp  
  add_mem'     := by  
    intro a b ha hb  
    simp at ha hb ; rw [ha, hb]  
  smul_mem'    := by  
    intro c p hp  
    change (c * p.1 = c * p.2)  
    rw [hp]
```

```
example (x : diagonalSubmodule) :  
  (x : α × α).1 = (x : α × α).2 :=
```

- ① carrier
- ② zero_mem'
- ③ add_mem'

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Module — Submodule subtype

```
def diagonalSubmodule :  
  Submodule ( × ) where  
  carrier      := {p | p.1 = p.2}  
  zero_mem'    := by simp  
  add_mem'     := by  
    intro a b ha hb  
    simp at ha hb ; rw [ha, hb]  
  smul_mem'    := by  
    intro c p hp  
    change (c * p.1 = c * p.2)  
    rw [hp]
```

```
example (x : diagonalSubmodule) :  
  (x : × ) 1 = (x : × ) 2 :=
```

- ① carrier
- ② zero_mem'
- ③ add_mem'
- ④ smul_mem'

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Module — Submodule subtype

```
def diagonalSubmodule :  
  Submodule ( × ) where  
  carrier      := {p | p.1 = p.2}  
  zero_mem'    := by simp  
  add_mem'     := by  
    intro a b ha hb  
    simp at ha hb ; rw [ha, hb]  
  smul_mem'    := by  
    intro c p hp  
    change (c * p.1 = c * p.2)  
    rw [hp]
```

```
example (x : diagonalSubmodule) :  
  (x : × ) 1 = (x : × ) 2 :=
```

- ① carrier
- ② zero_mem'
- ③ add_mem'
- ④ smul_mem'
- ⑤

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8
- 9 Prop-valued Typeclass

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Symplectic vector space —

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Symplectic vector space —

```
variable (F W : Type*)
variable [Field F] [Fintype F]
variable [AddCommGroup W] [Module
  F W]
variable [FiniteDimensional F W]

def IsAlternating
  ( : W → [F] W → [F] F) : Prop :=
  w : W,   w w = 0

def IsNondegenerate
  ( : W → [F] W → [F] F) : Prop :=
  w : W   ( v : W   w v = 0) → w =
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Symplectic vector space —

```
variable (F W : Type*)
variable [Field F] [Fintype F]
variable [AddCommGroup W] [Module
  F W]
variable [FiniteDimensional F W]

def IsAlternating
  ( : W → [F] W → [F] F) : Prop :=
  w : W,    w w = 0

def IsNondegenerate
  ( : W → [F] W → [F] F) : Prop :=
  w : W    ( v : W    w v = 0) → w =
```

AI

Lean

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Symplectic vector space —

```
variable (F W : Type*)
variable [Field F] [Fintype F]
variable [AddCommGroup W] [Module
  F W]
variable [FiniteDimensional F W]
```

```
def IsAlternating
  ( : W → [F] W → [F] F) : Prop :=
  w : W,    w w = 0
```

```
def IsNondegenerate
  ( : W → [F] W → [F] F) : Prop :=
  w : W    ( v : W    w v = 0) → w =
```

• W

AI

Lean


```

Prop-valued Typeclass
  Instance Search
    AI Lean
      AI
        Lean
          Finset
            SimpleGraph

```

```

variable (F W : Type*)
variable [Field F] [Fintype F]
variable [AddCommGroup W] [Module
  F W]
variable [FiniteDimensional F W]

def IsAlternating
  ( : W → [F] W → [F] F) : Prop :=
  w : W,   w w = 0

def IsNondegenerate
  ( : W → [F] W → [F] F) : Prop :=
  w : W     ( v : W     w v = 0) → v = 0

```

- AI Lean

AddGroup / Group
 Subtype
 Ring / CommRing
 Field
 Module / Submodule

Prop-valued Typeclass
 Instance Search
 AI Lean
 AI
 Lean
 Finset
 SimpleGraph

Symplectic vector space —

```
variable (F W : Type*)
variable [Field F] [Fintype F]
variable [AddCommGroup W] [Module
  F W]
variable [FiniteDimensional F W]

def IsAlternating
  ( : W → [F] W → [F] F) : Prop :=
  w : W,    w w = 0

def IsNondegenerate
  ( : W → [F] W → [F] F) : Prop :=
  w : W    ( v : W    w v = 0) → w =
```

- W
-
- Prop
-

AI

Lean

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

CompletePolarization

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

CompletePolarization

```
def TotallyIsotropic
  ( : W → [F] W → [F] F)
  (U : Submodule F W) : Prop :=
  u ∈ U, v ∈ U, u v = 0

structure CompletePolarization
  ( : W → [F] W → [F] F) where
  X      : Submodule F W
  Y      : Submodule F W
  isotropic_X : TotallyIsotropic F
    W X
  isotropic_Y : TotallyIsotropic F
    W Y
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

CompletePolarization

```
def TotallyIsotropic
  ( : W → [F] W → [F] F)
  (U : Submodule F W) : Prop :=
  u ∈ U, v ∈ U, u v = 0

structure CompletePolarization
  ( : W → [F] W → [F] F) where
  X      : Submodule F W
  Y      : Submodule F W
  isotropic_X : TotallyIsotropic F
    W X
  isotropic_Y : TotallyIsotropic F
    W Y
```

sup_eq_top w : W

AddGroup / Group
 Subtype
 Ring / CommRing
 Field
 Module / Submodule

Prop-valued Typeclass
 Instance Search
 AI Lean
 AI
 Lean
 Finset
 SimpleGraph

CompletePolarization

```
def TotallyIsotropic
  ( : W → [F] W → [F] F)
  (U : Submodule F W) : Prop :=
  u ∈ U, v ∈ U, u v = 0
```

```
structure CompletePolarization
  ( : W → [F] W → [F] F) where
  X      : Submodule F W
  Y      : Submodule F W
  isotropic_X : TotallyIsotropic F
    W X
  isotropic_Y : TotallyIsotropic F
    W Y
```

• X, Y

sup_eq_top w : W

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

CompletePolarization

```
def TotallyIsotropic
  ( : W → [F] W → [F] F)
  (U : Submodule F W) : Prop :=
  u U, v U, u v = 0

structure CompletePolarization
  ( : W → [F] W → [F] F) where
  X : Submodule F W
  Y : Submodule F W
  isotropic_X : TotallyIsotropic F
    W X
  isotropic_Y : TotallyIsotropic F
    W Y
```

- X, Y
-
- Disjoint X Y

sup_eq_top w : W

AddGroup / Group
 Subtype
 Ring / CommRing
 Field
 Module / Submodule

Prop-valued Typeclass
 Instance Search
 AI Lean
 AI
 Lean
 Finset
 SimpleGraph

CompletePolarization

```
def TotallyIsotropic
  ( : W → [F] W → [F] F)
  (U : Submodule F W) : Prop :=
  u U, v U, u v = 0

structure CompletePolarization
  ( : W → [F] W → [F] F) where
  X      : Submodule F W
  Y      : Submodule F W
  isotropic_X : TotallyIsotropic F
    W X
  isotropic_Y : TotallyIsotropic F
    W Y
```

- X, Y
-
- Disjoint X Y
- X Y =

sup_eq_top w : W

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Classical.choose —

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Classical.choose —

```
#check Classical.choose
-- (h : x, p x) →
#check Classical.choose_spec
-- p (Classical.choose h)

lemma exists_decomposition
  (P : CompletePolarization F W )
  (w : W) :
  x P.X, y P.Y, w = x + y :=
by sorry -- exercise: proof
  deferred
```

```
noncomputable def xComponent
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Classical.choose —

```
#check Classical.choose
-- (h : x, p x) →
#check Classical.choose_spec
-- p (Classical.choose h)

lemma exists_decomposition
  (P : CompletePolarization F W )
  (w : W) :
    x P.X, y P.Y, w = x + y :=
  by sorry -- exercise: proof
    deferred
```

```
noncomputable def xComponent
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Classical.choose —

```
#check Classical.choose
-- (h : x, p x) →
#check Classical.choose_spec
-- p (Classical.choose h)

lemma exists_decomposition
  (P : CompletePolarization F W )
  (w : W) :
    x P.X, y P.Y, w = x + y :=
  by sorry -- exercise: proof
    deferred
```

• $x, p\ x$

" "

noncomputable def xComponent

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Classical.choose —

```
#check Classical.choose
-- (h : x, p x) →
#check Classical.choose_spec
-- p (Classical.choose h)

lemma exists_decomposition
  (P : CompletePolarization F W )
  (w : W) :
    x P.X, y P.Y, w = x + y :=
  by sorry -- exercise: proof
    deferred
```

- $x, p\ x$
- choose witness

" "

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Classical.choose —

```
#check Classical.choose
-- (h : x, p x) →
#check Classical.choose_spec
-- p (Classical.choose h)

lemma exists_decomposition
  (P : CompletePolarization F W )
  (w : W) :
    x P.X, y P.Y, w = x + y :=
  by sorry -- exercise: proof
    deferred
```

```
noncomputable def xComponent
```

- $x, p\ x$
- choose witness
- choose_spec

" "

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Classical.choose —

```
#check Classical.choose
-- (h : x, p x) →
#check Classical.choose_spec
-- p (Classical.choose h)

lemma exists_decomposition
  (P : CompletePolarization F W )
  (w : W) :
  x P.X, y P.Y, w = x + y :=
by sorry -- exercise: proof
  deferred
```

```
noncomputable def xComponent
```

- $x, p\ x$
- choose witness
- choose_spec
- noncomputable

" "

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

choose_spec

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

choose_spec

```
noncomputable def yComponent
  (P : CompletePolarization F W )
  (w : W) : W :=
Classical.choose
  (Classical.choose_spec
    (exists_decomposition F W P
      w)).2
```

```
lemma decompose_by_components
  (P : CompletePolarization F W )
  (w : W) :
w = xComponent F W P w
  + yComponent F W P w := by
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

choose_spec

```
noncomputable def yComponent
  (P : CompletePolarization F W )
  (w : W) : W :=
Classical.choose
  (Classical.choose_spec
    (exists_decomposition F W P
      w)).2
```

```
lemma decompose_by_components
  (P : CompletePolarization F W )
  (w : W) :
w = xComponent F W P w
  + yComponent F W P w := by
```

Lean witness
choose_spec

```

Prop-valued Typeclass
  Instance Search
    AI Lean
      AI
        Lean
          Finset
            SimpleGraph

```

```

noncomputable def yComponent
  (P : CompletePolarization F W )
  (w : W) : W :=
Classical.choose
  (Classical.choose_spec
    (exists_decomposition F W P
      w)).2

```

```

lemma decompose_by_components
  (P : CompletePolarization F W )
  (w : W) :
  w = xComponent F W P w
    + yComponent F W P w := by

```

A set of small navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide controls.

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8
- 9 Prop-valued Typeclass

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Category

— Mathlib

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Category — Mathlib

```
class CategoryStruct (obj :  
  Type u)  
  extends Quiver obj where  
  id : X : obj, Hom X X  
  comp : {X Y Z : obj},  
    (X → Y) → (Y → Z) → (X → Z)  
class Category (obj : Type u)  
  extends CategoryStruct  
    obj where  
  id_comp : {X Y} (f : X  
    → Y),  
    X → f = f  
  comp_id : {X Y} (f : X
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Category — Mathlib

```
class CategoryStruct (obj :  
  Type u)  
  extends Quiver obj where  
  id : X : obj, Hom X X  
  comp : {X Y Z : obj},  
    (X → Y) → (Y → Z) → (X → Z)  
class Category (obj : Type u)  
  extends CategoryStruct  
    obj where  
  id_comp : {X Y} (f : X  
    → Y),  
    X → f = f  
  comp_id : {X Y} (f : X
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Category — Mathlib

```
class CategoryStruct (obj :  
  Type u)  
  extends Quiver obj where  
  id : X : obj, Hom X X  
  comp : {X Y Z : obj},  
    (X → Y) → (Y → Z) → (X → Z)  
class Category (obj : Type u)  
  extends CategoryStruct  
    obj where  
  id_comp : {X Y} (f : X  
    → Y),  
    X → f = f  
  comp_id : {X Y} (f : X
```

• Hom Quiver

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Category — Mathlib

```
class CategoryStruct (obj :  
  Type u)  
  extends Quiver obj where  
  id : X : obj, Hom X X  
  comp : {X Y Z : obj},  
    (X → Y) → (Y → Z) → (X → Z)  
class Category (obj : Type u)  
  extends CategoryStruct  
    obj where  
  id_comp : {X Y} (f : X  
    → Y),  
    X → f = f  
  comp_id : {X Y} (f : X
```

- Hom Quiver
- $X \text{ id } X$

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Category — Mathlib

```
class CategoryStruct (obj :  
  Type u)  
  extends Quiver obj where  
  id : X : obj, Hom X X  
  comp : {X Y Z : obj},  
    (X → Y) → (Y → Z) → (X → Z)  
class Category (obj : Type u)  
  extends CategoryStruct  
    obj where  
  id_comp : {X Y} (f : X  
    → Y),  
    X → f = f  
  comp_id : {X Y} (f : X
```

- Hom Quiver
- $X \text{ id } X$
- $f \circ g$

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Category — Mathlib

```
class CategoryStruct (obj :  
  Type u)  
  extends Quiver obj where  
  id : X : obj, Hom X X  
  comp : {X Y Z : obj},  
    (X → Y) → (Y → Z) → (X → Z)  
class Category (obj : Type u)  
  extends CategoryStruct  
    obj where  
  id_comp : {X Y} (f : X  
    → Y),  
    X → f = f  
  comp_id : {X Y} (f : X
```

- Hom Quiver
- $X \rightarrow id X$
- $f \circ g$
- instance

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Category — Mathlib

```
class CategoryStruct (obj :  
  Type u)  
  extends Quiver obj where  
  id : X : obj, Hom X X  
  comp : {X Y Z : obj},  
    (X → Y) → (Y → Z) → (X → Z)  
class Category (obj : Type u)  
  extends CategoryStruct  
    obj where  
  id_comp : {X Y} (f : X  
    → Y),  
    X → f = f  
  comp_id : {X Y} (f : X
```

- Hom Quiver
- $X \rightarrow id X$
- $f \circ g$
- instance
-

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Functor

— Mathlib

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Functor — Mathlib

```
structure Functor (C : Type u)
  [Category C]
  (D : Type u) [Category D]
  where
    obj : C → D
    map : {X Y : C},
      (X → Y) → (obj X → obj Y)
    map_id : X → obj X,
      map (id : X → X) = id
    map_comp : f → g,
      map (f → g) = map f ∘ map g
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Functor — Mathlib

```
structure Functor (C : Type u)
  [Category C]
  (D : Type u) [Category D]
  where
    obj : C → D
    map : {X Y : C},
      (X → Y) → (obj X → obj Y)
    map_id : X → obj X,
      map (id : X → X) = id
    map_comp : f → g,
      map (f → g) = map f ∘ map g
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Functor — Mathlib

```
structure Functor (C : Type u)
  [Category C]
  (D : Type u) [Category D]
  where
    obj : C → D
    map : {X Y : C},
      (X → Y) → (obj X → obj Y)
    map_id : X → obj X,
      map (id : X → X) = id
    map_comp : f g,
      map (f ∘ g) = map f ∘ map g
```

• $F.obj \ X$

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Functor — Mathlib

```
structure Functor (C : Type u)
  [Category C]
  (D : Type u) [Category D]
  where
    obj : C → D
    map : {X Y : C},
      (X → Y) → (obj X → obj Y)
    map_id : X → obj X,
      map (id : X → X) = id
    map_comp : f g,
      map (f ∘ g) = map f ∘ map g
```

- `F.obj X`
- `F.map f`

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Functor — Mathlib

```
structure Functor (C : Type u)
  [Category C]
  (D : Type u) [Category D]
  where
    obj : C → D
    map : {X Y : C},
      (X → Y) → (obj X → obj Y)
    map_id : X → obj X,
      map (id : X → X) = id
    map_comp : f g,
      map (f ∘ g) = map f ∘ map g
```

- `F.obj X`
- `F.map f`
- `F` `G`

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Functor — Mathlib

```
structure Functor (C : Type u)
  [Category C]
  (D : Type u) [Category D]
  where
    obj : C → D
    map : {X Y : C},
      (X → Y) → (obj X → obj Y)
    map_id : 1_X,
      map (1_X) = 1_(obj X)
    map_comp : f ∘ g,
      map (f ∘ g) = map f ∘ map g
```

- $F.obj\ X$
- $F.map\ f$
- $F\ C \rightarrow D$
- $F : C \rightarrow D$

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

NatTrans

— Mathlib

Mathlib

morphism

: $F \rightarrow G$

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

NatTrans — Mathlib

```
structure NatTrans (F G : C
  D) where
  app (X : C) : F.obj X
    G.obj X
  naturality X Y : C (f : X
    Y) :
    F.map f app Y =
    app X G.map f
```

Mathlib morphism : F G

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

NatTrans — Mathlib

```
structure NatTrans (F G : C
  D) where
  app (X : C) : F.obj X
    G.obj X
  naturality X Y : C (f : X
    Y) :
    F.map f app Y =
    app X G.map f
```

Mathlib morphism : F G

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

NatTrans — Mathlib

```
structure NatTrans (F G : C
  D) where
  app (X : C) : F.obj X
    G.obj X
  naturality X Y : C (f : X
    Y) :
    F.map f app Y =
    app X G.map f
```

• .app X

Mathlib morphism : F G

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

NatTrans — Mathlib

```
structure NatTrans (F G : C
  D) where
  app (X : C) : F.obj X
  G.obj X
  naturality X Y : C (f : X
    Y) :
    F.map f app Y =
    app X G.map f
```

- .app X
-

Mathlib morphism : F G

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

NatTrans — Mathlib

```
structure NatTrans (F G : C
  D) where
  app (X : C) : F.obj X
  G.obj X
  naturality X Y : C (f : X
    Y) :
    F.map f app Y =
    app X G.map f
```

• .app X

•

•

Mathlib morphism : F G

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

HomologicalComplex

— Mathlib

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

HomologicalComplex — Mathlib

```
structure HomologicalComplex
  (c : ComplexShape) where
  X :  $\rightarrow V$ 
  d :  $i \rightarrow j, X \ i \rightarrow X \ j$ 
  shape :  $i \rightarrow j,$ 
     $\neg c.Rel \ i \ j \rightarrow d \ i \ j = 0$ 
  d_comp_d' :  $i \rightarrow j \rightarrow k,$ 
     $c.Rel \ i \ j \rightarrow c.Rel \ j \ k \rightarrow$ 
       $d \ i \ j \circ d \ j \ k = 0$ 
```

ComplexShape.up

HomologicalComplex.d_comp_d

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

HomologicalComplex

— Mathlib

```
structure HomologicalComplex
  (c : ComplexShape) where
  X :  $\rightarrow V$ 
  d :  $i \rightarrow j, X \ i \rightarrow X \ j$ 
  shape :  $i \rightarrow j,$ 
     $\neg c.Rel \ i \ j \rightarrow d \ i \ j = 0$ 
  d_comp_d' :  $i \rightarrow j \rightarrow k,$ 
     $c.Rel \ i \ j \rightarrow c.Rel \ j \ k \rightarrow$ 
       $d \ i \ j \circ d \ j \ k = 0$ 
```

ComplexShape.up

HomologicalComplex.d_comp_d



AddGroup / Group
 Subtype
 Ring / CommRing
 Field
 Module / Submodule

Prop-valued Typeclass
 Instance Search
 AI Lean
 AI
 Lean
 Finset
 SimpleGraph

HomologicalComplex

— Mathlib

```
structure HomologicalComplex
  (c : ComplexShape) where
  X :  $\rightarrow V$ 
  d :  $i \rightarrow j, X \ i \rightarrow X \ j$ 
  shape :  $i \rightarrow j,$ 
     $\neg c.Rel \ i \ j \rightarrow d \ i \ j = 0$ 
  d_comp_d' :  $i \rightarrow j \rightarrow k,$ 
     $c.Rel \ i \ j \rightarrow c.Rel \ j \ k \rightarrow$ 
       $d \ i \ j \circ d \ j \ k = 0$ 
```

• $X \ i \rightarrow i$

ComplexShape.up

HomologicalComplex.d_comp_d

AddGroup / Group
 Subtype
 Ring / CommRing
 Field
 Module / Submodule

Prop-valued Typeclass
 Instance Search
 AI Lean
 AI
 Lean
 Finset
 SimpleGraph

HomologicalComplex — Mathlib

```
structure HomologicalComplex
  (c : ComplexShape) where
  X :  $\rightarrow V$ 
  d :  $i \rightarrow j, X \ i \rightarrow X \ j$ 
  shape :  $i \rightarrow j$ ,
     $\neg c.Rel \ i \ j \rightarrow d \ i \ j = 0$ 
  d_comp_d' :  $i \rightarrow j \rightarrow k$ ,
     $c.Rel \ i \ j \rightarrow c.Rel \ j \ k \rightarrow$ 
       $d \ i \ j \circ d \ j \ k = 0$ 
```

- $X \ i \rightarrow V$
- $d \ i \rightarrow j, X \ i \rightarrow X \ j$

AddGroup / Group
 Subtype
 Ring / CommRing
 Field
 Module / Submodule

Prop-valued Typeclass
 Instance Search
 AI Lean
 AI
 Lean
 Finset
 SimpleGraph

HomologicalComplex — Mathlib

```
structure HomologicalComplex
  (c : ComplexShape) where
  X :  $\rightarrow V$ 
  d :  $i \rightarrow j, X \ i \rightarrow X \ j$ 
  shape :  $i \rightarrow j$ ,
     $\neg c.Rel \ i \ j \rightarrow d \ i \ j = 0$ 
  d_comp_d' :  $i \rightarrow j \rightarrow k$ ,
     $c.Rel \ i \ j \rightarrow c.Rel \ j \ k \rightarrow$ 
       $d \ i \ j \circ d \ j \ k = 0$ 
```

- $X \ i \rightarrow V$
- $d \ i \rightarrow j, X \ i \rightarrow X \ j$
- $shape : i \rightarrow j, \neg c.Rel \ i \ j \rightarrow d \ i \ j = 0$

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

HomologicalComplex — Mathlib

```
structure HomologicalComplex
  (c : ComplexShape) where
  X :  $\rightarrow V$ 
  d :  $i \rightarrow j, X \ i \rightarrow X \ j$ 
  shape :  $i \rightarrow j$ ,
     $\neg c.Rel \ i \ j \rightarrow d \ i \ j = 0$ 
  d_comp_d' :  $i \rightarrow j \rightarrow k$ ,
     $c.Rel \ i \ j \rightarrow c.Rel \ j \ k \rightarrow$ 
       $d \ i \ j \circ d \ j \ k = 0$ 
```

- $X \ i \ i$
- $d \ i \ j$
- shape
- d_comp_d'

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8
- 9 Prop-valued Typeclass

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Category instance

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Category instance

```
open CategoryTheory
inductive OneObj where
  | star
instance : Category OneObj where
  Hom _ _ := PUnit
  id _ := PUnit.unit
  comp _ _ := PUnit.unit
  id_comp := by
    intro X Y f; cases f; rfl
  comp_id := by
    intro X Y f; cases f; rfl
  assoc := by
    intro W X Y Z f g h
```


AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Category instance

```
open CategoryTheory
inductive OneObj where
  | star
instance : Category OneObj where
  Hom _ _ := PUnit
  id _ := PUnit.unit
  comp _ _ := PUnit.unit
  id_comp := by
    intro X Y f; cases f; rfl
  comp_id := by
    intro X Y f; cases f; rfl
  assoc := by
    intro W X Y Z f g h
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Category instance

```
open CategoryTheory
inductive OneObj where
  | star
instance : Category OneObj where
  Hom _ _ := PUnit
  id _ := PUnit.unit
  comp _ _ := PUnit.unit
  id_comp := by
    intro X Y f; cases f; rfl
  comp_id := by
    intro X Y f; cases f; rfl
  assoc := by
    intro W X Y Z f g h
```

① Hom/id/comp

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Category instance

```
open CategoryTheory
inductive OneObj where
  | star
instance : Category OneObj where
  Hom _ _ := PUnit
  id _ := PUnit.unit
  comp _ _ := PUnit.unit
  id_comp := by
    intro X Y f; cases f; rfl
  comp_id := by
    intro X Y f; cases f; rfl
  assoc := by
    intro W X Y Z f g h
```

1 Hom/id/comp

2

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Category instance

```
open CategoryTheory
inductive OneObj where
  | star
instance : Category OneObj where
  Hom _ _ := PUnit
  id _ := PUnit.unit
  comp _ _ := PUnit.unit
  id_comp := by
    intro X Y f; cases f; rfl
  comp_id := by
    intro X Y f; cases f; rfl
  assoc := by
    intro W X Y Z f g h
```

① Hom/id/comp

②

③

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

HomologicalComplex

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

HomologicalComplex

```
open CategoryTheory
open CategoryTheory.Limits
noncomputable def tinyZeroComplex :
  HomologicalComplex
    (Discrete PUnit)
    (ComplexShape.up ) where
X _ := Discrete.mk PUnit.unit
d _ _ := 0
shape := by
  intro i j hij; rfl
d_comp_d' := by
  intro i j k hij hjk; simp
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

HomologicalComplex

```
open CategoryTheory
open CategoryTheory.Limits
noncomputable def tinyZeroComplex :
  HomologicalComplex
    (Discrete PUnit)
    (ComplexShape.up ) where
X _ := Discrete.mk PUnit.unit
d _ _ := 0
shape := by
  intro i j hij; rfl
d_comp_d' := by
  intro i j k hij hjk; simp
```


AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

HomologicalComplex

```
open CategoryTheory
open CategoryTheory.Limits
noncomputable def tinyZeroComplex :
  HomologicalComplex
    (Discrete PUnit)
    (ComplexShape.up ) where
X _ := Discrete.mk PUnit.unit
d _ _ := 0
shape := by
  intro i j hij; rfl
d_comp_d' := by
  intro i j k hij hjk; simp
```

• Discrete PUnit

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

HomologicalComplex

```
open CategoryTheory
open CategoryTheory.Limits
noncomputable def tinyZeroComplex :
  HomologicalComplex
    (Discrete PUnit)
    (ComplexShape.up ) where
X _ := Discrete.mk PUnit.unit
d _ _ := 0
shape := by
  intro i j hij; rfl
d_comp_d' := by
  intro i j k hij hjk; simp
```

- Discrete PUnit
- 0

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

HomologicalComplex

```
open CategoryTheory
open CategoryTheory.Limits
noncomputable def tinyZeroComplex :
  HomologicalComplex
    (Discrete PUnit)
    (ComplexShape.up ) where
X _ := Discrete.mk PUnit.unit
d _ _ := 0
shape := by
  intro i j hij; rfl
d_comp_d' := by
  intro i j k hij hjk; simp
```

- Discrete PUnit
- 0
- shape

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

HomologicalComplex

```
open CategoryTheory
open CategoryTheory.Limits
noncomputable def tinyZeroComplex :
  HomologicalComplex
    (Discrete PUnit)
    (ComplexShape.up ) where
X _ := Discrete.mk PUnit.unit
d _ _ := 0
shape := by
  intro i j hij; rfl
d_comp_d' := by
  intro i j k hij hjk; simp
```

- Discrete PUnit
- 0
- shape
- simp

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass

Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8
- 9 Prop-valued Typeclass

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass

Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

instance

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass

Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

instance

instance

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

instance

instance

- Type

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

instance

instance

- Type
-

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

instance

instance

- Type
-
- Group G, Field K

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

instance

instance

- Type
-
- Group G, Field K
-

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

instance

instance

- Type
-
- Group G, Field K
-

instance

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

instance

instance

- Type
-
- Group G, Field K
-

instance

- Prop

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

instance

instance

- Type
-
- Group G, Field K
-

instance

- Prop
-

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

instance

instance

- Type
-
- Group G, Field K
-

instance

- Prop
-
- Fact P, [Decidable P]

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

instance

instance

- Type
-
- Group G, Field K
-

instance

- Prop
-
- Fact P, [Decidable P]
-

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass

Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Fact P typeclass search

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Fact P typeclass search

```
-- Fact P wraps a proof of P as an
  instance
-- class Fact (P : Prop) : Prop
  where
--   out : P

-- Usage: bring a hypothesis into
  instance search
example (P : Prop) [Fact P] : P :=
  Fact.out

-- haveI temporarily introduces an
  instance
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Fact P typeclass search

```
-- Fact P wraps a proof of P as an
  instance
-- class Fact (P : Prop) : Prop
  where
--   out : P

-- Usage: bring a hypothesis into
  instance search
example (P : Prop) [Fact P] : P :=
  Fact.out

-- haveI temporarily introduces an
  instance
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Fact P typeclass search

```
-- Fact P wraps a proof of P as an
  instance
-- class Fact (P : Prop) : Prop
  where
--   out : P

-- Usage: bring a hypothesis into
  instance search
example (P : Prop) [Fact P] : P :=
  Fact.out

-- haveI temporarily introduces an
  instance
```

① Fact P P

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Fact P typeclass search

```
-- Fact P wraps a proof of P as an
  instance
-- class Fact (P : Prop) : Prop
  where
--   out : P

-- Usage: bring a hypothesis into
  instance search
example (P : Prop) [Fact P] : P :=
  Fact.out

-- haveI temporarily introduces an
  instance
```

① Fact P P

② Fact.out

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Fact P typeclass search

```
-- Fact P wraps a proof of P as an
  instance
-- class Fact (P : Prop) : Prop
  where
--   out : P

-- Usage: bring a hypothesis into
  instance search
example (P : Prop) [Fact P] : P :=
  Fact.out

-- haveI temporarily introduces an
  instance
```

- 1 Fact P P
- 2 Fact.out
- 3 [Fact ...]

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Fact P typeclass search

```
-- Fact P wraps a proof of P as an
  instance
-- class Fact (P : Prop) : Prop
  where
--   out : P

-- Usage: bring a hypothesis into
  instance search
example (P : Prop) [Fact P] : P :=
  Fact.out

-- haveI temporarily introduces an
  instance
```

- 1 Fact P P
- 2 Fact.out
- 3 [Fact ...]
- 4 haveI

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass

Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

class

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

class

```
class IsDivisibleBy (n k : ) :  
  Prop where  
  witness : k n  
  
-- Use an instance implicitly  
example (n k : ) [IsDivisibleBy n  
  k] :  
  k n :=  
  IsDivisibleBy.witness  
  
-- inferInstance lets Lean search  
  for the proof  
example (n k : ) [h :
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

class

```
class IsDivisibleBy (n k : ) :  
  Prop where  
  witness : k n  
  
-- Use an instance implicitly  
example (n k : ) [IsDivisibleBy n  
  k] :  
  k n :=  
  IsDivisibleBy.witness  
  
-- inferInstance lets Lean search  
  for the proof  
example (n k : ) [h :
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

class

```
class IsDivisibleBy (n k : ) :  
  Prop where  
  witness : k n  
  
-- Use an instance implicitly  
example (n k : ) [IsDivisibleBy n  
  k] :  
  k n :=  
  IsDivisibleBy.witness  
  
-- inferInstance lets Lean search  
  for the proof  
example (n k : ) [h :
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

class

```
class IsDivisibleBy (n k : ) :  
  Prop where  
  witness : k n  
  
-- Use an instance implicitly  
example (n k : ) [IsDivisibleBy n  
  k] :  
  k n :=  
  IsDivisibleBy.witness  
  
-- inferInstance lets Lean search  
  for the proof  
example (n k : ) [h :
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

class

```
class IsDivisibleBy (n k : ) :  
  Prop where  
  witness : k n  
  
-- Use an instance implicitly  
example (n k : ) [IsDivisibleBy n  
  k] :  
  k n :=  
  IsDivisibleBy.witness  
  
-- inferInstance lets Lean search  
  for the proof  
example (n k : ) [h :
```

-
-
- inferInstance
Lean

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

class

```
class IsDivisibleBy (n k : ) :  
  Prop where  
  witness : k n  
  
-- Use an instance implicitly  
example (n k : ) [IsDivisibleBy n  
  k] :  
  k n :=  
  IsDivisibleBy.witness  
  
-- inferInstance lets Lean search  
  for the proof  
example (n k : ) [h :
```

-
-
- inferInstance
Lean
-

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8
- 9 Prop-valued Typeclass

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

instance search

#synth	infer_instance	haveI
Lean	proof	

-- Check that an instance exists

```
#synth Group Sign
```

-- Use it inside a proof

```
example : Group Sign := infer_instance
```

-- Temporarily introduce an instance

```
example (G : Type*) [Group G]  
[h : CommGroup G] (a b : G) :
```


AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass

Instance Search

AI Lean
AI
Lean
Finset
SimpleGraph

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

```
-- mul_assoc needs only Semigroup
#check @mul_assoc
--   {} [Semigroup ] (a b c : ),
--   a * b * c = a * (b * c)

-- inv_mul_cancel needs Group
#check @inv_mul_cancel
--   {G} [Group G] (a : G),
--   a1 * a = 1

-- group tactic works for any Group
example {G : Type*} [Group G]
  (a b : G) :
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

```
-- mul_assoc needs only Semigroup
#check @mul_assoc
--   {} [Semigroup ] (a b c : ),
--   a * b * c = a * (b * c)

-- inv_mul_cancel needs Group
#check @inv_mul_cancel
--   {G} [Group G] (a : G),
--   a1 * a = 1

-- group tactic works for any Group
example {G : Type*} [Group G]
  (a b : G) :
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

```
-- mul_assoc needs only Semigroup
#check @mul_assoc
--   {} [Semigroup ] (a b c : ),
--   a * b * c = a * (b * c)

-- inv_mul_cancel needs Group
#check @inv_mul_cancel
--   {G} [Group G] (a : G),
--   a1 * a = 1

-- group tactic works for any Group
example {G : Type*} [Group G]
  (a b : G) :
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

```
-- mul_assoc needs only Semigroup
#check @mul_assoc
--   {} [Semigroup ] (a b c : ),
--   a * b * c = a * (b * c)

-- inv_mul_cancel needs Group
#check @inv_mul_cancel
--   {G} [Group G] (a : G),
--   a-1 * a = 1

-- group tactic works for any Group
example {G : Type*} [Group G]
  (a b : G) :
```

-
- [Group G]
[CommGroup G]

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

```
-- mul_assoc needs only Semigroup
#check @mul_assoc
--   {} [Semigroup ] (a b c : ),
--   a * b * c = a * (b * c)

-- inv_mul_cancel needs Group
#check @inv_mul_cancel
--   {G} [Group G] (a : G),
--   a-1 * a = 1

-- group tactic works for any Group
example {G : Type*} [Group G]
  (a b : G) :
```

-
- [Group G]
[CommGroup G]
- AI

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

```
-- mul_assoc needs only Semigroup
#check @mul_assoc
--   {} [Semigroup ] (a b c : ),
--   a * b * c = a * (b * c)

-- inv_mul_cancel needs Group
#check @inv_mul_cancel
--   {G} [Group G] (a : G),
--   a1 * a = 1

-- group tactic works for any Group
example {G : Type*} [Group G]
  (a b : G) :
```

-
- [Group G]
[CommGroup G]
- AI
- #check

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— Equiv instance

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— Equiv instance

```
-- Transfer CommRing along an
  equivalence
def equivProd : PairInt → Int × Int
  := ...
instance : CommRing PairInt :=
  equivProd.commRing

-- Transfer Field along an
  equivalence
def equivRat : WrappedQ → ... := ...
instance : Field WrappedQ :=
  equivRat.field
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— Equiv instance

```
-- Transfer CommRing along an
  equivalence
def equivProd : PairInt → Int × Int
  := ...
instance : CommRing PairInt :=
  equivProd.commRing

-- Transfer Field along an
  equivalence
def equivRat : WrappedQ → ... := ...
instance : Field WrappedQ :=
  equivRat.field
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— Equiv instance

```
-- Transfer CommRing along an
  equivalence
def equivProd : PairInt → Int × Int
  := ...
instance : CommRing PairInt :=
  equivProd.commRing

-- Transfer Field along an
  equivalence
def equivRat : WrappedQ := ...
instance : Field WrappedQ :=
  equivRat.field
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— Equiv instance

```
-- Transfer CommRing along an
  equivalence
def equivProd : PairInt → Int × Int
  := ...
instance : CommRing PairInt :=
  equivProd.commRing

-- Transfer Field along an
  equivalence
def equivRat : WrappedQ := ...
instance : Field WrappedQ :=
  equivRat.field
```

equivalence

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— Equiv instance

```
-- Transfer CommRing along an
  equivalence
def equivProd : PairInt → Int × Int
  := ...
instance : CommRing PairInt :=
  equivProd.commRing

-- Transfer Field along an
  equivalence
def equivRat : WrappedQ := ...
instance : Field WrappedQ :=
  equivRat.field
```

-
- equivalence
- wrapper subtype

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— Equiv instance

```
-- Transfer CommRing along an
  equivalence
def equivProd : PairInt → Int × Int
  := ...
instance : CommRing PairInt :=
  equivProd.commRing

-- Transfer Field along an
  equivalence
def equivRat : WrappedQ := ...
instance : Field WrappedQ :=
  equivRat.field
```

-
- equivalence
- wrapper subtype
- AI Lean

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8
- 9 Prop-valued Typeclass

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

statement

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

statement

```
-- Missing hypothesis: what if  $x$   
= 0?
```

```
example (K : Type*) [Field K] (x :  
  K) :  
  x / x = 1 := by  
  field_simp          -- fails!
```

```
-- Add the necessary assumption
```

```
example (K : Type*) [Field K]  
  (x : K) (hx : x ≠ 0) :  
  x / x = 1 := by  
  field_simp [hx]
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

statement

```
--    Missing hypothesis: what if  $x$   
--    = 0?
```

```
example (K : Type*) [Field K] (x :  
  K) :  
  x / x = 1 := by  
  field_simp          -- fails!
```

```
--    Add the necessary assumption
```

```
example (K : Type*) [Field K]  
  (x : K) (hx : x ≠ 0) :  
  x / x = 1 := by  
  field_simp [hx]
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

statement

```
-- Missing hypothesis: what if x  
= 0?
```

```
example (K : Type*) [Field K] (x :  
  K) :  
  x / x = 1 := by  
  field_simp          -- fails!
```

```
-- Add the necessary assumption
```

```
example (K : Type*) [Field K]  
  (x : K) (hx : x ≠ 0) :  
  x / x = 1 := by  
  field_simp [hx]
```

$$\bullet x = 0$$

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

statement

```
-- Missing hypothesis: what if  $x$   
= 0?
```

```
example (K : Type*) [Field K] (x :  
  K) :  
  x / x = 1 := by  
  field_simp          -- fails!
```

```
-- Add the necessary assumption
```

```
example (K : Type*) [Field K]  
  (x : K) (hx : x ≠ 0) :  
  x / x = 1 := by  
  field_simp [hx]
```

- $x = 0$
- Lean

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

statement

```
-- Missing hypothesis: what if x  
= 0?
```

```
example (K : Type*) [Field K] (x :  
  K) :  
  x / x = 1 := by  
  field_simp          -- fails!
```

```
-- Add the necessary assumption
```

```
example (K : Type*) [Field K]  
  (x : K) (hx : x ≠ 0) :  
  x / x = 1 := by  
  field_simp [hx]
```

- $x = 0$
- Lean
- tactic

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

statement

```
--      Missing hypothesis: what if  $x$   
--      = 0?
```

```
example (K : Type*) [Field K] (x :  
  K) :  
  x / x = 1 := by  
  field_simp           -- fails!
```

```
--      Add the necessary assumption
```

```
example (K : Type*) [Field K]  
  (x : K) (hx : x ≠ 0) :  
  x / x = 1 := by  
  field_simp [hx]
```

- $x = 0$
- Lean
- tactic
- statement proof

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

```
--      False in a non-commutative
--      group!
example (G : Type*) [Group G] (a b
  : G) :
  (a * b) 1 = a 1 * b 1 := by
  group      -- fails: not provable

--      Correct for any Group
example (G : Type*) [Group G] (a b
  : G) :
  (a * b) 1 = b 1 * a 1 := by
  group
```


AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

```
--      False in a non-commutative
--      group!
example (G : Type*) [Group G] (a b
  : G) :
  (a * b) ^ 1 = a ^ 1 * b ^ 1 := by
group      -- fails: not provable

--      Correct for any Group
example (G : Type*) [Group G] (a b
  : G) :
  (a * b) ^ 1 = b ^ 1 * a ^ 1 := by
group
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

```
--      False in a non-commutative
      group!
example (G : Type*) [Group G] (a b
  : G) :
  (a * b) ^ 1 = a ^ 1 * b ^ 1 := by
group      -- fails: not provable

--      Correct for any Group
example (G : Type*) [Group G] (a b
  : G) :
  (a * b) ^ 1 = b ^ 1 * a ^ 1 := by
group
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

```
--      False in a non-commutative
      group!
example (G : Type*) [Group G] (a b
  : G) :
  (a * b) 1 = a1 * b1 := by
  group      -- fails: not provable

--      Correct for any Group
example (G : Type*) [Group G] (a b
  : G) :
  (a * b) 1 = b1 * a1 := by
  group
```



• group

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

```
--      False in a non-commutative
      group!
example (G : Type*) [Group G] (a b
  : G) :
  (a * b) 1 = a1 * b1 := by
  group      -- fails: not provable

--      Correct for any Group
example (G : Type*) [Group G] (a b
  : G) :
  (a * b) 1 = b1 * a1 := by
  group
```

-
- group
- [CommGroup G]

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

```
--      False in a non-commutative
      group!
example (G : Type*) [Group G] (a b
  : G) :
  (a * b) ^ 1 = a ^ 1 * b ^ 1 := by
  group      -- fails: not provable

--      Correct for any Group
example (G : Type*) [Group G] (a b
  : G) :
  (a * b) ^ 1 = b ^ 1 * a ^ 1 := by
  group
```

-
- group
- [CommGroup G]
-

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

API —

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

API —

```
--      This theorem name does not
      exist
example (n : ) :
  (Finset.range n).card = n := by
    exact Finset.card_range_eq_n --
      Error!

--      Check the real API first
#check Finset.card_range
-- Finset.card_range :
--      (n : ), (Finset.range
      n).card = n
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

API —

```
--      This theorem name does not
      exist
example (n : ) :
  (Finset.range n).card = n := by
    exact Finset.card_range_eq_n --
      Error!

--      Check the real API first
#check Finset.card_range
-- Finset.card_range :
--      (n : ), (Finset.range
      n).card = n
```


AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

API —

```
--      This theorem name does not
      exist
example (n : ) :
  (Finset.range n).card = n := by
    exact Finset.card_range_eq_n --
      Error!

--      Check the real API first
#check Finset.card_range
-- Finset.card_range :
--      (n : ), (Finset.range
      n).card = n
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

API —

```
--      This theorem name does not
      exist
example (n : ) :
  (Finset.range n).card = n := by
    exact Finset.card_range_eq_n --
      Error!

--      Check the real API first
#check Finset.card_range
-- Finset.card_range :
--      (n : ), (Finset.range
      n).card = n
```

API

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

API —

```
--      This theorem name does not
      exist
example (n : ) :
  (Finset.range n).card = n := by
    exact Finset.card_range_eq_n --
      Error!

--      Check the real API first
#check Finset.card_range
-- Finset.card_range :
--      (n : ), (Finset.range
      n).card = n
```

-
- API
- AI #check

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

API —

```
--      This theorem name does not
      exist
example (n : ) :
  (Finset.range n).card = n := by
    exact Finset.card_range_eq_n --
      Error!

--      Check the real API first
#check Finset.card_range
-- Finset.card_range :
--      (n : ), (Finset.range
      n).card = n
```

-
- API
- AI #check
- LeanSearch / Loogle

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

decide

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

decide

```
--      decide requires [Decidable P]
example (P : Prop) : P  ¬ P := by
  decide      -- Error: no Decidable
               instance
```

```
--      Constructive: add [Decidable
P]
example (P : Prop) [Decidable P] :
  P  ¬ P := by
  by_cases h : P
  · exact Or.inl h
  · exact Or.inr h
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

decide

```
--      decide requires [Decidable P]
example (P : Prop) : P  ¬ P := by
  decide      -- Error: no Decidable
               instance

--      Constructive: add [Decidable
P]
example (P : Prop) [Decidable P] :
  P  ¬ P := by
  by_cases h : P
  · exact Or.inl h
  · exact Or.inr h
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

decide

```
--      decide requires [Decidable P]
example (P : Prop) : P  ¬ P := by
  decide      -- Error: no Decidable
               instance

--      Constructive: add [Decidable
P]
example (P : Prop) [Decidable P] :
  P  ¬ P := by
  by_cases h : P
  · exact Or.inl h
  · exact Or.inr h
```

❶ decide
[Decidable P]

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

decide

```
--      decide requires [Decidable P]
example (P : Prop) : P  ¬ P := by
  decide      -- Error: no Decidable
               instance
```

```
--      Constructive: add [Decidable
P]
example (P : Prop) [Decidable P] :
  P  ¬ P := by
  by_cases h : P
  · exact Or.inl h
  · exact Or.inr h
```

- 1 decide
[Decidable P]
- 2 decide

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

decide

```
--      decide requires [Decidable P]
example (P : Prop) : P  ¬ P := by
  decide      -- Error: no Decidable
               instance

--      Constructive: add [Decidable
P]
example (P : Prop) [Decidable P] :
  P  ¬ P := by
  by_cases h : P
  · exact Or.inl h
  · exact Or.inr h
```

- 1 decide
[Decidable P]
- 2 decide
- 3 classical

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

decide

```
--      decide requires [Decidable P]
example (P : Prop) : P  ¬ P := by
  decide      -- Error: no Decidable
              instance

--      Constructive: add [Decidable
P]
example (P : Prop) [Decidable P] :
  P  ¬ P := by
  by_cases h : P
  · exact Or.inl h
  · exact Or.inr h
```

- 1 decide
[Decidable P]
- 2 decide
- 3 classical
- 4 AI

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8
- 9 Prop-valued Typeclass

AddGroup / Group
 Subtype
 Ring / CommRing
 Field
 Module / Submodule

Prop-valued Typeclass
 Instance Search
 AI Lean
AI
 Lean
 Finset
 SimpleGraph

AI Lean

	AI	Lean
Statement	typeclass	
Search	#check	theorem
Proof	proof	lemma
Review		simp

AI Lean kernel

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

AI prompt

Prompt

Lean 4 / Mathlib
theorem statement

1. typeclass
- 2.
3. #check
4. theorem
5. statement

AddGroup / Group
 Subtype
 Ring / CommRing
 Field
 Module / Submodule

Prop-valued Typeclass
 Instance Search
 AI Lean
AI
 Lean
 Finset
 SimpleGraph

Lean

	$x / x = 1$	$x = 0$	AI
	Group	CommGroup	#check
API	theorem		#check
			Mathlib
	tactic		lemma proof

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8
- 9 Prop-valued Typeclass

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Lean — Kernel elaborator

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Lean — Kernel elaborator

Lean

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Lean — Kernel elaborator

Lean

- theorem statement

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Lean — Kernel elaborator

Lean

- theorem statement
- proof term

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Lean — Kernel elaborator

Lean

- theorem statement
- proof term
- instance

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Lean — Kernel elaborator

Lean

- theorem statement
- proof term
- instance
-

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Lean — Kernel elaborator

Lean

- theorem statement
- proof term
- instance
-

Lean

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Lean — Kernel elaborator

Lean

- theorem statement
- proof term
- instance
-

Lean

-

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Lean — Kernel elaborator

Lean

- theorem statement
- proof term
- instance
-

Lean

-
- Mathlib

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Lean — Kernel elaborator

Lean

- theorem statement
- proof term
- instance
-

Lean

-
- Mathlib
-

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Lean — Kernel elaborator

Lean

- theorem statement
- proof term
- instance
-

Lean

-
- Mathlib
-
-

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8
- 9 Prop-valued Typeclass

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Finset

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Finset

Finset α

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Finset

Finset α



AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Finset

Finset α

-
- card

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Finset

Finset α

-
- `card`
- `range n` $\{0, \dots, n - 1\}$

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Finset

Finset α

-
- card
- range n $\{0, \dots, n - 1\}$
- filter

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Finset

Finset α

-
- card
- range n $\{0, \dots, n - 1\}$
- filter

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Finset

Finset α

-
- `card`
- `range n` $\{0, \dots, n - 1\}$
- `filter`

- `#check`
`Finset.card_range`

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Finset

Finset α

-
- `card`
- `range n` $\{0, \dots, n - 1\}$
- `filter`
- `#check`
`Finset.card_range`
- `simp`

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Finset

Finset α

-
- `card`
- `range n` $\{0, \dots, n - 1\}$
- `filter`
- `#check`
`Finset.card_range`
- `simp`
- `native_decide`

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Finset

Finset α

-
- `card`
- `range n` $\{0, \dots, n - 1\}$
- `filter`
- `#check`
`Finset.card_range`
- `simp`
- `native_decide`
- `sum_range_succ`

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Finset

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Finset

```
open BigOperators Finset

-- card of range n is n (general)
#check Finset.card_range
-- (n : ℕ), (Finset.range n).card
--   = n
example (n : ℕ) :
  (Finset.range n).card = n :=
  Finset.card_range n

-- constant sum over range
-- (general)
example (n : ℕ) :
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Finset

```
open BigOperators Finset

-- card of range n is n (general)
#check Finset.card_range
-- (n : ℕ), (Finset.range n).card
--   = n
example (n : ℕ) :
  (Finset.range n).card = n :=
  Finset.card_range n

-- constant sum over range
-- (general)
example (n : ℕ) :
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Finset

```
open BigOperators Finset
```

```
-- card of range n is n (general)  
#check Finset.card_range  
-- (n : ℕ), (Finset.range n).card  
--   = n
```

```
example (n : ℕ) :  
  (Finset.range n).card = n :=  
  Finset.card_range n
```

```
-- constant sum over range  
-- (general)
```

```
example (n : ℕ) :
```

• API

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Finset

```
open BigOperators Finset
```

```
-- card of range n is n (general)  
#check Finset.card_range  
-- (n : ℕ), (Finset.range n).card  
--   = n
```

```
example (n : ℕ) :  
  (Finset.range n).card = n :=  
  Finset.card_range n
```

```
-- constant sum over range  
-- (general)
```

```
example (n : ℕ) :
```

• API

•

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Finset

```
open BigOperators Finset

-- card of range n is n (general)
#check Finset.card_range
-- (n : ℕ), (Finset.range n).card
  = n
example (n : ℕ) :
  (Finset.range n).card = n :=
  Finset.card_range n

-- constant sum over range
  (general)
example (n : ℕ) :
```

• API



AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Finset

```
open BigOperators Finset

-- card of range n is n (general)
#check Finset.card_range
-- (n : ℕ), (Finset.range n).card
  = n
example (n : ℕ) :
  (Finset.range n).card = n :=
  Finset.card_range n

-- constant sum over range
  (general)
example (n : ℕ) :
```

• API



AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Finset

n

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Finset n

```
-- General theorem (Gauss sum)
theorem sum_range_id (n : ℕ) :
  ∑ i in Finset.range (n + 1), i =
    n * (n + 1) / 2 := by
  induction n with
  | zero => simp
  | succ n ih =>
    rw [Finset.sum_range_succ, +
        ih]
  ring
```

```
-- Verify the identity holds
```

```
#check Finset.sum_range_id
```


AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Finset n

```
-- General theorem (Gauss sum)
theorem sum_range_id (n : ℕ) :
  ∑ i in Finset.range (n + 1), i =
    n * (n + 1) / 2 := by
  induction n with
  | zero => simp
  | succ n ih =>
    rw [Finset.sum_range_succ, +
        ih]
  ring
```

```
-- Verify the identity holds
#check Finset.sum_range_id
```

Finset

tactic

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Finset n

```
-- General theorem (Gauss sum)
theorem sum_range_id (n : ℕ) :
  ∑ i in Finset.range (n + 1), i =
    n * (n + 1) / 2 := by
  induction n with
  | zero => simp
  | succ n ih =>
    rw [Finset.sum_range_succ, +
        ih]
  ring
```

```
-- Verify the identity holds
#check Finset.sum_range_id
```

1 sum_range_succ

Finset

tactic

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Finset n

```
-- General theorem (Gauss sum)
theorem sum_range_id (n : ℕ) :
  ∑ i in Finset.range (n + 1), i =
    n * (n + 1) / 2 := by
  induction n with
  | zero => simp
  | succ n ih =>
    rw [Finset.sum_range_succ, +
        ih]
  ring
```

```
-- Verify the identity holds
#check Finset.sum_range_id
```

① sum_range_succ

②

Finset

tactic

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Finset n

```
-- General theorem (Gauss sum)
theorem sum_range_id (n : ℕ) :
  ∑ i in Finset.range (n + 1), i =
    n * (n + 1) / 2 := by
  induction n with
  | zero => simp
  | succ n ih =>
    rw [Finset.sum_range_succ, +
        ih]
    ring
```

```
-- Verify the identity holds
#check Finset.sum_range_id
```

- 1 sum_range_succ
- 2
- 3 ring

Finset

tactic

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Finset n

```
-- General theorem (Gauss sum)
theorem sum_range_id (n : ℕ) :
  ∑ i in Finset.range (n + 1), i =
    n * (n + 1) / 2 := by
  induction n with
  | zero => simp
  | succ n ih =>
    rw [Finset.sum_range_succ, +
        ih]
    ring
```

```
-- Verify the identity holds
#check Finset.sum_range_id
```

1 sum_range_succ

2

3 ring

4

Finset

tactic

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8
- 9 Prop-valued Typeclass

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

— SimpleGraph = +

Lean

```
G : SimpleGraph V V
G.Adj u v      u v
symm
loopless      G.Adj v v
V = Fin n      fin_cases
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

```
def path3 : SimpleGraph (Fin 3)
  where
  Adj i j :=
    (i = 0   j = 1)  (i = 1   j =
      0)
    (i = 1   j = 2)  (i = 2   j = 1)
  symm := by
    intro i j h
    fin_cases i <;> fin_cases j <;>
    simp_all
  loopless := by
    intro i h
    fin_cases i <;> simp_all
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

```
def path3 : SimpleGraph (Fin 3)
  where
  Adj i j :=
    (i = 0   j = 1)  (i = 1   j =
      0)
    (i = 1   j = 2)  (i = 2   j = 1)
  symm := by
    intro i j h
    fin_cases i <;> fin_cases j <;>
    simp_all
  loopless := by
    intro i h
    fin_cases i <;> simp_all
```


AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

```
def path3 : SimpleGraph (Fin 3)
  where
  Adj i j :=
    (i = 0   j = 1)  (i = 1   j =
      0)
    (i = 1   j = 2)  (i = 2   j = 1)
  symm := by
    intro i j h
    fin_cases i <.> fin_cases j <.>
    simp_all
  loopless := by
    intro i h
    fin_cases i <.> simp_all
```

- 0, 1, 2
- 0--1 1--2

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

```
def path3 : SimpleGraph (Fin 3)
  where
  Adj i j :=
    (i = 0  j = 1)  (i = 1  j =
      0)
    (i = 1  j = 2)  (i = 2  j = 1)
  symm := by
    intro i j h
    fin_cases i <;> fin_cases j <;>
    simp_all
  loopless := by
    intro i h
    fin_cases i <;> simp_all
```

- 0, 1, 2
- 0--1 1--2
- symm

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

```
def path3 : SimpleGraph (Fin 3)
  where
  Adj i j :=
    (i = 0   j = 1)  (i = 1   j =
      0)
    (i = 1   j = 2)  (i = 2   j = 1)
  symm := by
    intro i j h
    fin_cases i <;> fin_cases j <;>
    simp_all
  loopless := by
    intro i h
    fin_cases i <;> simp_all
```

- 0, 1, 2
- 0--1 1--2
- symm
- loopless

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

SimpleGraph

simp

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

SimpleGraph

simp

```
-- General: adjacency implies
distinct vertices
example (V : Type*) (G :
  SimpleGraph V)
  (u v : V) (h : G.Adj u v) : u
    v :=
G.ne_of_adj h

-- Concrete: path3 adjacency
example : path3.Adj 0 1 := by simp
[path3]
example : path3.Adj 1 2 := by simp
[path3]
```


AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

SimpleGraph

simp

```
-- General: adjacency implies
distinct vertices
example (V : Type*) (G :
  SimpleGraph V)
  (u v : V) (h : G.Adj u v) : u
    v :=
G.ne_of_adj h

-- Concrete: path3 adjacency
example : path3.Adj 0 1 := by simp
[ path3 ]
example : path3.Adj 1 2 := by simp
[ path3 ]
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

SimpleGraph

simp

```
-- General: adjacency implies  
distinct vertices
```

```
example (V : Type*) (G :  
  SimpleGraph V)  
  (u v : V) (h : G.Adj u v) : u  
    v :=  
  G.ne_of_adj h
```

```
-- Concrete: path3 adjacency
```

```
example : path3.Adj 0 1 := by simp  
[path3]
```

```
example : path3.Adj 1 2 := by simp  
[path3]
```

1 Adj

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

SimpleGraph

simp

```
-- General: adjacency implies
distinct vertices
example (V : Type*) (G :
  SimpleGraph V)
  (u v : V) (h : G.Adj u v) : u
    v :=
G.ne_of_adj h

-- Concrete: path3 adjacency
example : path3.Adj 0 1 := by simp
[path3]
example : path3.Adj 1 2 := by simp
[path3]
```

1 Adj

2

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

SimpleGraph

simp

```
-- General: adjacency implies
distinct vertices
example (V : Type*) (G :
  SimpleGraph V)
  (u v : V) (h : G.Adj u v) : u
    v :=
G.ne_of_adj h

-- Concrete: path3 adjacency
example : path3.Adj 0 1 := by simp
[path3]
example : path3.Adj 1 2 := by simp
[path3]
```

1 Adj

2

3

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

SimpleGraph

simp

```
-- General: adjacency implies
distinct vertices
example (V : Type*) (G :
  SimpleGraph V)
  (u v : V) (h : G.Adj u v) : u
    v :=
G.ne_of_adj h

-- Concrete: path3 adjacency
example : path3.Adj 0 1 := by simp
[path3]
example : path3.Adj 1 2 := by simp
[path3]
```

- 1 Adj
- 2
- 3
- 4 simp / decide

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

SimpleGraph

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

SimpleGraph

```
-- ne_of_adj is a general theorem
#check @SimpleGraph.ne_of_adj
-- {V} (G : SimpleGraph V) {u v :
  V},
--   G.Adj u v → u = v

-- Conjunction of adjacency facts
example :
  path3.Adj 0 1 → path3.Adj 0 2
  :=
  by simp [path3], by simp [path3]
```


AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

SimpleGraph

```
-- ne_of_adj is a general theorem
#check @SimpleGraph.ne_of_adj
-- {V} (G : SimpleGraph V) {u v :
  V},
-- G.Adj u v → u ~ v
```

```
-- Conjunction of adjacency facts
example :
  path3.Adj 0 1 ~ path3.Adj 0 2
  :=
  by simp [path3], by simp [path3]
```

```
-- Complete graph: u ~ v ↔ Adj u v
```

• ne_of_adj

" " " "

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

SimpleGraph

```
-- ne_of_adj is a general theorem
#check @SimpleGraph.ne_of_adj
-- {V} (G : SimpleGraph V) {u v :
  V},
--   G.Adj u v → u ~ v

-- Conjunction of adjacency facts
example :
  path3.Adj 0 1 ~ path3.Adj 0 2
  :=
  by simp [path3], by simp [path3]
```

- ne_of_adj
-
- $u \sim v \rightarrow \text{Adj } u \sim v$

" " " "

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

SimpleGraph

```
-- ne_of_adj is a general theorem
#check @SimpleGraph.ne_of_adj
-- {V} (G : SimpleGraph V) {u v :
  V},
--   G.Adj u v → u ~ v

-- Conjunction of adjacency facts
example :
  path3.Adj 0 1 ~ path3.Adj 0 2
  :=
  by simp [path3], by simp [path3]
```

```
• ne_of_adj
•
•   u ~ v ~ path3.Adj u
    v
•
•   " " " "
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8
- 9 Prop-valued Typeclass

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

instance

1

G / R / K / C

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

instance

- 1 $G / R / K / C$
- 2 AddGroup Group CommRing Field Module Category

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

instance

- ① `G / R / K / C`
- ② `AddGroup Group CommRing Field Module Category`
- ③ `#print`

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

instance

- ① `G / R / K / C`
- ② `AddGroup Group CommRing Field Module Category`
- ③ `#print`
- ④ `carrier`

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

instance

- ① `G / R / K / C`
- ② `AddGroup Group CommRing Field Module Category`
- ③ `#print`
- ④ `carrier`
- ⑤ `add/mul/zero/one/smul/Hom/id/comp`

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

instance

```
①      G / R / K / C
②      AddGroup Group CommRing Field Module Category
③      #print
④      carrier
⑤      add/mul/zero/one/smul/Hom/id/comp
⑥      #synth
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

instance

```
①      G / R / K / C
②      AddGroup Group CommRing Field Module Category
③      #print
④      carrier
⑤      add/mul/zero/one/smul/Hom/id/comp
⑥      #synth
⑦      #check
```

AddGroup / Group
Subtype
Ring / CommRing
Field
Module / Submodule

Prop-valued Typeclass
Instance Search
AI Lean
AI
Lean
Finset
SimpleGraph

instance

```
1      G / R / K / C
2      AddGroup Group CommRing Field Module Category
3  #print
4      carrier
5      add/mul/zero/one/smul/Hom/id/comp
6  #synth
7  #check
8  AI      Lean
```